
Exact Learning of Arithmetic with Differentiable Agents

Hristo Papazov
TML Lab, EPFL
hristo.papazov@epfl.ch

Francesco D’Angelo
TML Lab, EPFL
francesco.dangelo@epfl.ch

Nicolas Flammarion
TML Lab, EPFL
nicolas.flammarion@epfl.ch

Abstract

We explore the possibility of exact algorithmic learning with gradient-based methods and introduce a differentiable framework capable of strong length generalization on arithmetic tasks. Our approach centers on Differentiable Finite-State Transducers (DFSTs), a Turing-complete model family that avoids the pitfalls of prior architectures by enabling constant-precision, constant-time generation, and end-to-end log-parallel differentiable training. Leveraging policy-trajectory observations from expert agents, we train DFSTs to perform binary and decimal addition and multiplication. Remarkably, models trained on tiny datasets generalize without error to inputs thousands of times longer than the training examples. These results show that training differentiable agents on structured intermediate supervision could pave the way towards exact gradient-based learning of algorithmic skills.

1 Introduction

The dream of AGI envisions systems capable of exact learning of algorithmic skills – such as searching, sorting, or arithmetic – from a reasonable dataset of examples [38, 9, 17]. Here, *exact learning* refers to the criterion proposed by Angluin [3] which requires correct application of a ground-truth rule to every possible input. More formally, given a finite problem alphabet Σ and an unknown model M^* computing some function $f^* : \mathcal{D} \rightarrow \Sigma^*$ over an input domain $\mathcal{D} \subseteq \Sigma^*$, the exact learning paradigm requires a learner $\mathcal{L}$ to identify f^* with probability 1 after observing some part of M^* ’s computation on finitely many inputs $z_1, \dots, z_N \in \mathcal{D}$.

In this paper, we entertain the idea of gradient-based exact learning and provide as evidence a simple differentiable agent capable of unprecedented state-of-the-art length generalization on arithmetic tasks. In pursuing the ambitious goal of differentiable exact learning, a couple of considerations arise.

Training Data. First, we cannot expect our differentiable learner $\mathcal{L}$ to identify every computable function f^* from input-output examples $(z, f^*(z))$ alone. Indeed, Gold [13] showed that no algorithm can exactly learn the class of recursive functions in the limit from input-output examples (Theorem I.5), highlighting the inadequacy of such training data. Consequently, we adopt the formal framework of Papazov and Flammarion [31] for learning algorithms in the limit from observations of intermediate steps. As we will soon discuss, the addition of observable intermediate steps, termed *policy-trajectory observations* (PTOs), allows us to reduce the intractable problem of learning recursive functions to the more manageable one of learning finite-state transducers (FSTs) [31, see Theorem 16].

Model Class. Second, since for any computable f^* , we want gradient-based search to find some differentiable model capable of executing f^* , we need to ensure the computational universality (or Turing-completeness) of our model class. To the best of our knowledge, all previously proposed universal model families commit at least one of the following two unforgivable sins for optimization:

- **Exploding Computation Time:** RNNs and Neural GPUs with unbounded precision of internal arithmetic [37, 33]; Transformers with growing context windows and unbounded/scaling internal arithmetic [33, 28, 5]; recurrent models with growing differentially referenceable memories [16];
- **Uninformative Gradients:** Architectures with non-differentiable adaptive-computation-time mechanisms [14, 11]; models interacting with an external environment without differentiable (or even continuous) feedback [21, 10, 34].

These caveats hurt gradient-based optimization. The first not only causes an inference-time explosion but also increases the computational distance between emitting an output and receiving a gradient signal. The second produces gradients with dubious optimization information in a discontinuous loss landscape. Thus, the cited works achieve Turing-completeness at the cost of rendering gradient-based training impossible. In contrast, by allowing our Differentiable Finite-State Transducer (DFST) family free motion over an external environment, we addresses all three limitations and present a framework with constant precision, constant-time output generation, computational universality, and the capacity for end-to-end differentiable training.

Note that we omit any optimization critique of other neurosymbolic architectures such as Neural Turing Machines [15], Neural GPUs [22], Neural Random-Access Machines [24], Pointer Networks [40], Hierarchical Attentive Memories [2], finite-precision recurrent networks [42, 27], finite-precision Transformers [33, 18, 29], SSMs [30], and Hierarchical Reasoning Models [41] since these model families lack computational universality and therefore cannot, in principle, exactly learn arbitrary algorithms. Moreover, most of the listed architectures can only express linear memory algorithms.

Main Contributions. Our paper introduces a simple, differentiable, Turing-complete, and parallelizable setup consisting of a DFST agent which interacts with an external environment through action tokens. We train our DFST model on observed computational traces (PTOs) from expert agents performing four different arithmetic tasks: binary addition and multiplication (`add2`, `mult2`) and decimal addition and multiplication (`add10`, `mult10`) of two numbers. For `add2` and `add10`, we train on tiny datasets consisting of 20 and 225 examples, respectively, featuring summands with at most 3 digits. Training on these datasets led to both robust and probabilistic length generalization (RLG and PLG)¹ of 3850 and 2450 digits, respectively. Testing for correctness beyond these numbers of digits leads to `OutOfMemory` errors on our A100-SXM4-80GB GPU. Similarly, for `mult2` and `mult10`, we train on datasets consisting of 750 and 10000 samples, respectively, featuring multiplicands with up to 5 digits. Training on these datasets led to RLG and PLG of 600 and 180 digits, respectively, and we could not test on larger numbers due to insufficient memory. In particular, as far as we tested, we could not find a single error in the computation of our best trained models for each arithmetic task. Moreover, we achieved this state-of-the-art length generalization with a well-motivated scratchpad data in the form of PTOs and a straightforward training procedure.

Table 1: Neural GPU vs. DFST on Arithmetic Tasks

Framework	Metric	<code>add2</code>	<code>add10</code>	<code>mult2</code>	<code>mult10</code>
Neural GPU w/ Input-Output Data	# Samples	~ 200k	N/A	~ 200k	N/A
	# Model Parameters	10368	N/A	10368	N/A
	Max Train Number Length	20	N/A	20	N/A
	Robust LG	≤ 20	N/A	≤ 8	N/A
	Probabilistic LG	$\geq \mathbf{2000}$	N/A	$\geq \mathbf{2000}$	N/A
DFST (ours) w/ PTO Data	# Samples	20	225	750	10000
	# Model Parameters	1020	8900	5280	162108
	Max Train Number Length	3	3	5	5
	Robust LG	$\geq \mathbf{3850}$	$\geq \mathbf{2450}$	$\geq \mathbf{600}$	$\geq \mathbf{180}$
	Probabilistic LG	$\geq \mathbf{3850}$	$\geq \mathbf{2450}$	$\geq \mathbf{600}$	$\geq \mathbf{180}$

Bold **numbers** indicate inability to test on longer inputs due to GPU memory constraints.

Related Work. Since exact learning represents the limit of length generalization, we review in passing some experimental results on length generalization in sequence-to-sequence models. Recent empirical studies [21, 12, 26, 35, 23, 20] demonstrated that Transformers, modified RNNs, and

¹We formally define RLG and PLG for arithmetic tasks in Section 3.

LSTMs length-generalize poorly on a variety of simple algorithmic tasks when trained on input-output data. With additional data-formatting strategies – such as task-specific index hints, optimized positional encodings, input reversing, position coupling, and scratchpad guidance – multiple works on arithmetic generalization in Transformers [4, 20, 43, 25, 44, 19, 7, 8] achieved non-trivial 2–3x length generalization with high but not perfect exact-match accuracy (EMA).

Interestingly, similar to us, Hou et al. [19] train on scratchpad data consisting of the computational traces of a Turing machine (TM) designed to solve the specific algorithmic task. However, whereas our agent only observes the evolution of the TM tape, Hou et al. [19] train with access to the TM state transitions, effectively trivializing the task, as the transformer merely needs to memorize the given algorithm without performing any program synthesis.

Finally, we make a detailed comparison in Table 1 with the prior work [22, 32] most relevant to our paper. Kaiser and Sutskever [22] introduce a Neural Cellular Automaton, called the Neural GPU, which achieves remarkable PLG when trained solely on input-output examples of binary addition and multiplication. Unfortunately, Price et al. [32] report that the Neural GPU’s generalization capacity highly depends on the random seed used for training and initialization. Only a few random seeds and hyperparameter configurations lead to 2000-digit PLG, and all trained models fail on highly structured and symmetric examples containing only a few digits. In contrast, our trained models achieve perfect accuracy on long symmetric examples as far as we could test, and unlike the Neural GPU, also manage to learn the challenging decimal arithmetic tasks. Furthermore, our DFST training relies only on a simple cosine learning-rate scheduler, while the Neural GPU training depends on multiple intricate techniques, including curriculum learning, gradient-noise injection, gradient scheduling, relaxation-pull, dropout, and gate cut-off. Hence, as shown in Table 1, the use of observable intermediate steps increases the robustness and simplicity of the training procedure, greatly diminishes the number of training samples, and enables the learning of decimal arithmetic.

2 Learning Framework

Intelligent agents in the wild hardly ever learn novel algorithmic tasks from input-output observations. Instead, real-life learners observe computational processes and reconstruct the ground-truth function by making sense of the intermediate steps. We briefly restate the formalization of this idea due to Papazov and Flammarion [31] before adapting the framework to our setting. We start with the underlying environment, which we model as a symbolic universe.

Definition 2.1 (Symbolic Universe). Given an enumerable set G and a finite set of symbols Σ containing the empty symbol λ , a world-state $w : G \rightarrow \Sigma$ is a function with finitely many non-empty assignments. The symbolic universe $\mathcal{U} = \mathcal{U}(G, \Sigma)$ is the set of all such world-states.

The set G serves as the geometry of the universe. For our setup, we will only work on an unbounded 2-dimensional grid $G = \mathbb{Z}^2$ and observe the computation of *grid agents*.

Definition 2.2 (Grid Agent). A grid agent operating in a symbolic universe $\mathcal{U}(\mathbb{Z}^2, \Sigma)$ constitutes a triple $M = (Q, \mathcal{U}, \delta)$, where Q is a finite set of states and $\delta : Q \times \mathbb{Z}^2 \times \mathcal{U} \rightarrow Q \times \mathbb{Z}^2 \times \mathcal{U}$ is a computable transition function with perception restrictions given below.

The grid agent M evolves and interacts with the environment according to the deterministic transition rules $\delta(q, p, w) = (q', p', w')$, where $q \mapsto q' \in Q$ denotes the state update, $p \mapsto p' \in \mathbb{Z}^2$ – the shift in perception, and $w \mapsto w' \in \mathcal{U}$ – the change of the world-state. Importantly, **(i)** $\delta(q, p, w)$ depends only on the current state q and the observed cell $w(p)$; **(ii)** $\|p - p'\|_2 \leq 1$ – i.e., the agent moves *continuously* up (U), down (D), left (L), right (R), or stays (S); and **(iii)** M can only edit the world-state at position p . For a complete interaction trace, we specify an initial state $q_0 \in Q$, an initial perceived cell $p_0 \in \mathbb{Z}^2$, and a final state $q_f \in Q$ as a halting condition. Note that by setting $\mathcal{M} = \{U, D, L, R, S\}$, we can essentially redefine the transition function δ as a mapping from $Q \times \Sigma$ to $Q \times \Sigma \times \mathcal{M}$. Clearly, grid agents extend Turing machines to the plane, and hence, the family of grid agents $\mathcal{G}$ is computationally universal.

Grid agents also naturally model how human teachers navigate around a blackboard, with q corresponding to the teacher’s state of mind, p – the chalk’s position, and $w(p)$ and $w'(p)$ – the currently observed and written symbol, respectively. This analogy allows us to derive the policy-trajectory observations defined in [31]. Indeed, at time $t \in \mathbb{N}$, the students have no access to the teacher’s current neural state q_t but can still observe how the teacher M reacts with a symbol token $s_t = w_t(p_t) \in \Sigma$ and a motion token $m_t = “p_{t+1} - p_t” \in \mathcal{M}$ to the current partial observation

$x_t = w_t(p_t) \in \Sigma$. Thus, the students observe the trajectory $(x_t, a_t)_{t=0}^T$ of the history-dependent policy $\pi_M(x_0, \dots, x_t) = a_t = (s_t, m_t)$ enacted by the teacher – hence the name, PTOs.

In our code base, we constructed 4 grid agents $M_{\text{add}2}$, $M_{\text{add}10}$, $M_{\text{mult}2}$, $M_{\text{mult}10}$ emulating the aforementioned 4 arithmetic tasks and operating on 4 problem alphabets $\Sigma_{\text{add}2} = \{0, 1, \lambda, +\}$, $\Sigma_{\text{add}10} = \{0, \dots, 9, \lambda, +\}$, $\Sigma_{\text{mult}2} = \{0, 1, \lambda, \times\}$, $\Sigma_{\text{mult}10} = \{0, \dots, 9, \lambda, \times\}$. Each of these grid agents receives as input a string $z = a \circ b$ (where $\circ = +/\times$) horizontally written on the grid, while all other cells remain empty, and proceeds to compute, leaving only the final answer on the grid.

A Differentiable Learner. Papazov and Flammarion [31] showed that a computationally costly learning-by-enumeration strategy provably identifies any computational agent M^* in the limit from PTOs. In this paper, we consider a more efficient gradient-based approach. Namely, we assume that the ground-truth model M^* comes from the universal class $\mathcal{G}$ of grid agents and train a DFST (defined below), on a small dataset of computational traces from M^* .

Definition 2.3 (Differentiable Finite-State Transducer Agent). Given a symbol-token alphabet Σ and a motion-token alphabet $\mathcal{M}$, a DFST of dimension d operating over the symbolic grid $\mathcal{U}(\mathbb{Z}^2, \Sigma)$ with motion vocabulary $\mathcal{M}$ consists of 3 trainable weight tensors $A \in \mathbb{R}^{|\Sigma| \times d \times d}$, $B \in \mathbb{R}^{|\Sigma| \times |\Sigma| \times d}$, $C \in \mathbb{R}^{|\Sigma| \times |\mathcal{M}| \times d}$ and a trainable initial hidden state $h_0 \in \mathbb{R}^d$. Upon observing a one-hot symbol encoding $x_t \in \mathbb{R}^{|\Sigma|}$ at time $t \geq 0$, which corresponds to the currently perceived part of the world-state $w_t(p_t)$, the DFST performs the following updates:

$$\begin{aligned} h_{t+1} &= A(x_t)h_t \\ \hat{s}_t &= B(x_t)h_t \\ \hat{m}_t &= C(x_t)h_t, \end{aligned}$$

and outputs a symbol token $\sigma_t = \text{argmax}(\hat{s}_t) \in \Sigma$ and a motion token $\mu_t = \text{argmax}(\hat{m}_t) \in \mathcal{M}$, which change the world-state and shift the perception as in Figure 1.

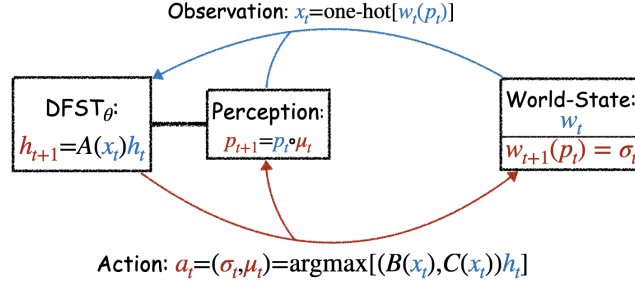


Figure 1: A DFST agent interacting with an external environment.

Let us denote by Δ_p the space of p -precision DFSTs. Let $\psi_p : \Delta_p \rightarrow \mathcal{G}$ denote the mapping that equates a DFST agent $M_\partial(A, B, C, h_0)$ with a grid agent $M(Q, \delta)$ by building the set of states Q from the 2^{pd} possible values for h_t and by constructing the transition function δ from the tensors A, B, C . Clearly, a DFST with a hidden-state dimension d can express the policy of any grid agent with at most d states by one-hot encoding the state transitions δ into the tensors A, B, C . In other words, if $M \in \mathcal{G}$ has d states $q_0, \dots, q_{d-1}$, Σ has k symbols $s_0, \dots, s_{k-1}$, and $\mathcal{M}$ has r motion tokens $m_0, \dots, m_{r-1}$, we let q_i, s_i , and m_i correspond to the basis vectors e_{i+1} in $\mathbb{R}^d, \mathbb{R}^k$, and $\mathbb{R}^r$, respectively. Then, we set $h_0^\delta = e_1$, and if $\delta(q_i, s_j) = (q_{i'}, s_{j'}, m_\ell)$, we set $A^\delta[j, i', i] = 1, B^\delta[j, j', i] = 1, C^\delta[j, \ell, i] = 1$, and we define all other tensor weights as 0. Thus, the updates issued by the DFST($A^\delta, B^\delta, C^\delta, h_0^\delta$) exactly emulate δ , and we arrive at the following theorem.

Theorem 2.1 (Universality of DFSTs). *The map $\psi_p : \Delta_p \rightarrow \mathcal{G}$ is surjective. In particular, for any precision $p \in \mathbb{N}$, the DFST family Δ_p is Turing-complete when interacting with an external symbolic grid $\mathcal{U}(\mathbb{Z}^2, \Sigma)$. Moreover, any d -state grid agent admits emulation by DFST agents of dimension d .*

The novelty in our proposed model family comes from the decoupling of the differentiable agent and the environment. Indeed, sequential models such as Transformers and RNNs move in a single direction with no means of controlling the next interaction spot with the environment. We amend this limitation by allowing our DFST control unit to not only edit the grid but also to issue commands for where the next interaction should occur.

We emphasize the point that any sequential neural network capable of state tracking (i.e., emulating a finite-state transducer) could, in principle, learn the policy of a grid agent $M^* \in \mathcal{G}$. Such models include RNNs, LSTMs, GRUs [1, 36, 42, 27, 39] but not Transformers [29] or diagonal and non-gated SSMs [30]. Nevertheless, we chose the DFST as our trainable model due to its linear structure and simplicity, which we hope will inspire future theoretical analysis. Moreover, since DFSTs contain no nonlinearities, parallel scans (à la Blelloch [6]) allow for **fast log-parallel training**.

3 Experimental Details

We train on PTOs from the expert grid agents $M_{\text{task}R}$ with the MSE loss $\frac{1}{2T} \sum_{t=0}^T [(\hat{s}_t - s_t)^2 + (\hat{m}_t - m_t)^2]$ under the next-action prediction (NAP) objective. Here, s_t and m_t denote the one-hot encodings of the symbol and motion tokens emitted by the expert models at time $t \geq 0$.

Identity Initialization. We train four DFST control units $D_{\text{add}2}$, $D_{\text{add}10}$, $D_{\text{mult}2}$, and $D_{\text{mult}10}$ with hidden-state dimensions matching the number of states of the corresponding ground-truth grid agent: i.e., 12, 20, 32, and 108. We initialize the state-transition matrices $A[i]$, $\forall i < |\Sigma|$, as the identity $I_d \in \mathbb{R}^{d \times d}$ and the tensors B and C as 0. We initialize each coordinate of h_0 uniformly at random and independently in the interval (0,1), after which we normalize h_0 to have a unit Euclidean norm.

Optimizer. We use the standard Adam optimizer with a cosine annealing scheduler and no warm-up period. We start the training of $D_{\text{add}10}$, $D_{\text{mult}2}$, and $D_{\text{mult}10}$ with a learning rate of 0.001, and the training of $D_{\text{add}2}$ – with a learning rate of 0.01. We always use batch size 32 and float32 precision.

Data Selection. We create a data sampling function $\text{DS}(R, \text{task}, p, q, N)$ which for radix R and arithmetic task task , samples all PTOs from $M_{\text{task}R}$ on number pairs (a, b) with at most p digits – that is, a total of $R^2(R^p - 1)^2/(R - 1)^2$ samples. Then, DS samples all R^2 pairs of q -digit numbers made of a single digit repeated q times. After that, if $N > R^2(R^p - 1)^2/(R - 1)^2 + R^2$, DS samples at random another $N - R^2(R^p - 1)^2/(R - 1)^2 + R^2$ unique pairs of numbers having up to q digits. For $\text{add}2$ we use $p = 1, q = 3, N = 20$, for $\text{add}10$ — $p = 1, q = 3, N = 225$, for $\text{mult}2$ — $p = 1, q = 5, N = 750$, and for $\text{mult}10$ — $p = 1, q = 5, N = 10000$.

Length Generalization. We define the Probabilistic Length Generalization (PLG) of a model as the largest number of digits m such that the model achieves perfect EMA when tested on 5 random pairs of numbers having exactly m digits and 5 random pairs of numbers having at most m digits. We define the Robust Length Generalization (RLG) of a model as the largest number of digits m such that $\text{PLG} \geq m$ and the model achieves perfect EMA on the R^2 pairs of m -digit numbers that have m identical digits. We noticed after extensive testing that (as observed by Price et al. [32]) same-digit numbers constitute the hard test instances for generalization. Clearly, $\text{RLG} \leq \text{PLG}$. In Section A, we show the relationship between longer training time and RLG for experiments with random seed 42. In Table 1, our reported values for PLG and RLG reflect our experiments on $\text{add}2$, $\text{add}10$, $\text{mult}2$, and $\text{mult}10$ for training lengths 500k, 500k, 3mln, and 3mln iterations, respectively.

Memory Constraints. During training, our DFST models observe PTOs of lengths at most 70 for the addition tasks and at most 464 for the multiplication tasks. However, the lengths of PTOs from $M_{\text{add}2}$, $M_{\text{add}10}$, $M_{\text{mult}2}$, $M_{\text{mult}10}$ on number pairs with 3850, 2450, 600, and 180 digits, respectively, become on the order of 30mln, 12mln, 6mln, and 500k operations. Checking the EMA on sequences above these lengths completely fills the memory of our A100-SXM4-80GB GPU.

Verifying Exact Learning. We draw attention to the fact that verifying whether two grid agents follow the same deterministic policy reduces to deciding the equivalence of two Turing machines. Since the language $\text{EQ}_{\text{TM}} = \{ \langle M_1, M_2 \rangle \mid L(M_1) = L(M_2) \}$ is undecidable, no general procedure can check whether $\pi_{M_{\text{task}R}} \equiv \pi_{D_{\text{task}R}}$ by just observing the descriptions of $M_{\text{task}R}$ and $D_{\text{task}R}$. Consequently, we rely on random tests to gain confidence in the ability of $D_{\text{task}R}$.

4 Conclusion

Our results demonstrate that Differentiable Finite-State Transducers achieve unprecedented length generalization on arithmetic tasks using simple, gradient-based training. This work provides a concrete step toward the broader goal of enabling differentiable agents to learn precise algorithmic behavior through structured intermediate supervision. We hope that our minimalist, differentiable, and Turing-complete framework will inspire further theoretical explorations of the loss landscape of algorithmic learning.

References

- [1] N. Alon, A. K. Dewdney, and T. J. Ott. Efficient simulation of finite automata by neural nets. *Journal of the ACM (JACM)*, 38(2):495–514, 1991.
- [2] M. Andrychowicz and K. Kurach. Learning efficient algorithms with hierarchical attentive memory. *arXiv preprint arXiv:1602.03218*, 2016.
- [3] D. Angluin. Queries and concept learning. *Machine learning*, 2(4):319–342, 1988.
- [4] C. Anil, Y. Wu, A. Andreassen, A. Lewkowycz, V. Misra, V. Ramasesh, A. Slone, G. Gur-Ari, E. Dyer, and B. Neyshabur. Exploring length generalization in large language models. *Advances in Neural Information Processing Systems*, 35:38546–38556, 2022.
- [5] S. Bhattamishra, A. Patel, and N. Goyal. On the computational power of transformers and its implications in sequence modeling. *arXiv preprint arXiv:2006.09286*, 2020.
- [6] G. E. Blelloch. Prefix sums and their applications. 1990.
- [7] H. Cho, J. Cha, P. Awasthi, S. Bhojanapalli, A. Gupta, and C. Yun. Position coupling: Improving length generalization of arithmetic transformers using task structure. *Advances in Neural Information Processing Systems*, 37:22233–22315, 2024.
- [8] H. Cho, J. Cha, S. Bhojanapalli, and C. Yun. Arithmetic transformers can length-generalize in both operand length and count. *arXiv preprint arXiv:2410.15787*, 2024.
- [9] F. Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- [10] S. Chung and H. Siegelmann. Turing completeness of bounded-precision recurrent neural networks. *Advances in neural information processing systems*, 34:28431–28441, 2021.
- [11] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018.
- [12] G. Delétang, A. Ruoss, J. Grau-Moya, T. Genewein, L. K. Wenliang, E. Catt, C. Cundy, M. Hutter, S. Legg, J. Veness, et al. Neural networks and the chomsky hierarchy. *arXiv preprint arXiv:2207.02098*, 2022.
- [13] E. M. Gold. Language identification in the limit. *Information and control*, 10(5):447–474, 1967.
- [14] A. Graves. Adaptive computation time for recurrent neural networks. *arXiv*, 2016.
- [15] A. Graves, G. Wayne, and I. Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [16] E. Grefenstette, K. M. Hermann, M. Suleyman, and P. Blunsom. Learning to transduce with unbounded memory. *Advances in neural information processing systems*, 28, 2015.
- [17] A. György, T. Lattimore, N. Lazić, and C. Szepesvári. Beyond statistical learning: Exact learning is essential for general intelligence. *arXiv preprint arXiv:2506.23908*, 2025.
- [18] M. Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020.
- [19] K. Hou, D. Brandfonbrener, S. Kakade, S. Jelassi, and E. Malach. Universal length generalization with turing programs. *arXiv preprint arXiv:2407.03310*, 2024.
- [20] S. Jelassi, S. d’Ascoli, C. Domingo-Enrich, Y. Wu, Y. Li, and F. Charton. Length generalization in arithmetic transformers. *arXiv preprint arXiv:2306.15400*, 2023.
- [21] A. Joulin and T. Mikolov. Inferring algorithmic patterns with stack-augmented recurrent nets. *Advances in neural information processing systems*, 28, 2015.
- [22] L. Kaiser and I. Sutskever. Neural gpus learn algorithms. *arXiv preprint arXiv:1511.08228*, 2015.

- [23] A. Kazemnejad, I. Padhi, K. Natesan Ramamurthy, P. Das, and S. Reddy. The impact of positional encoding on length generalization in transformers. *Advances in Neural Information Processing Systems*, 36:24892–24928, 2023.
- [24] K. Kurach, M. Andrychowicz, and I. Sutskever. Neural random-access machines. *arXiv preprint arXiv:1511.06392*, 2015.
- [25] N. Lee, K. Sreenivasan, J. D. Lee, K. Lee, and D. Papailiopoulos. Teaching arithmetic to small transformers. *arXiv preprint arXiv:2307.03381*, 2023.
- [26] B. Liu, J. T. Ash, S. Goel, A. Krishnamurthy, and C. Zhang. Transformers learn shortcuts to automata. *arXiv preprint arXiv:2210.10749*, 2022.
- [27] W. Merrill. Sequential neural networks as automata. *arXiv preprint arXiv:1906.01615*, 2019.
- [28] W. Merrill and A. Sabharwal. The expressive power of transformers with chain of thought. *CoRR*, 2023.
- [29] W. Merrill and A. Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023.
- [30] W. Merrill, J. Petty, and A. Sabharwal. The illusion of state in state-space models. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.
- [31] H. Papazov and N. Flammarion. Learning algorithms in the limit. In N. Haghtalab and A. Moitra, editors, *Proceedings of Thirty Eighth Conference on Learning Theory*, volume 291 of *Proceedings of Machine Learning Research*, pages 4486–4510. PMLR, 2025.
- [32] E. Price, W. Zaremba, and I. Sutskever. Extensions and limitations of the neural gpu. *arXiv preprint arXiv:1611.00736*, 2016.
- [33] J. Pérez, J. Marinković, and P. Barceló. On the turing completeness of modern neural network architectures. In *International Conference on Learning Representations*, 2019.
- [34] D. Schuurmans, H. Dai, and F. Zanini. Autoregressive large language models are computationally universal. *arXiv preprint arXiv:2410.03170*, 2024.
- [35] R. Shen, S. Bubeck, R. Eldan, Y. T. Lee, Y. Li, and Y. Zhang. Positional description matters for transformers arithmetic. *arXiv preprint arXiv:2311.14737*, 2023.
- [36] H. T. Siegelmann. Recurrent neural networks and finite automata. *Computational intelligence*, 12(4):567–574, 1996.
- [37] H. T. Siegelmann and E. D. Sontag. On the computational power of neural nets. In *Proceedings of the fifth annual workshop on Computational learning theory*, pages 440–449, 1992.
- [38] R. J. Solomonoff et al. Machine learning-past and future. *Dartmouth, NH, July*, 2006.
- [39] A. Svete and R. Cotterell. Recurrent neural language models as probabilistic finite-state automata. *arXiv preprint arXiv:2310.05161*, 2023.
- [40] O. Vinyals, M. Fortunato, and N. Jaitly. Pointer networks. *Advances in neural information processing systems*, 28, 2015.
- [41] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. A. Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.
- [42] G. Weiss, Y. Goldberg, and E. Yahav. On the practical computational power of finite precision rnns for language recognition. *arXiv preprint arXiv:1805.04908*, 2018.
- [43] H. Zhou, A. Bradley, E. Littwin, N. Razin, O. Saremi, J. Susskind, S. Bengio, and P. Nakkiran. What algorithms can transformers learn? a study in length generalization. *arXiv preprint arXiv:2310.16028*, 2023.
- [44] Y. Zhou, U. Alon, X. Chen, X. Wang, R. Agarwal, and D. Zhou. Transformers can achieve length generalization but not robustly. *arXiv preprint arXiv:2402.09371*, 2024.

A Robust Length Generalization vs. Training Time

In this section we report the experimental results that highlight the trade-off between robust length generalization and training time across various model sizes and training tasks. Our findings indicate that increased training times correlate with lower training loss and improved length generalization capabilities.

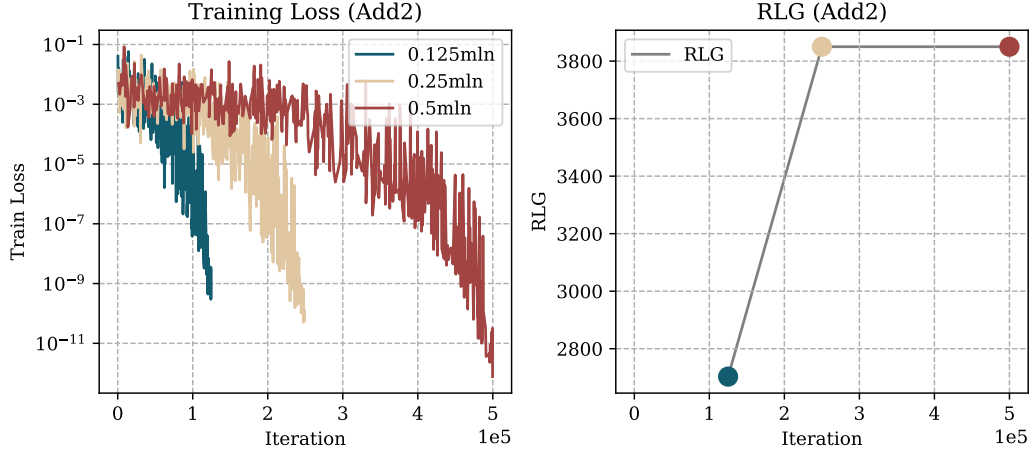


Figure 2: **Binary addition:** Training loss (left) and RLG (robust length generalization, right) across training iterations.

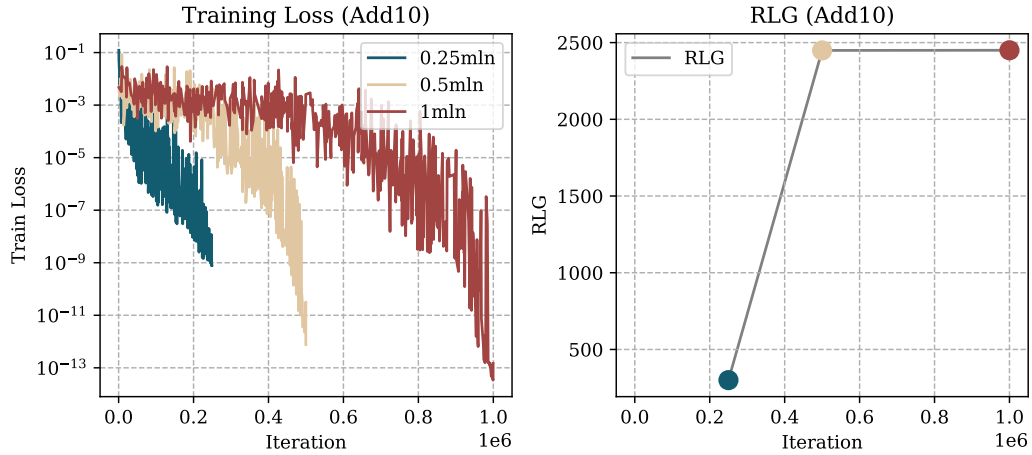


Figure 3: **Decimal addition:** Training loss (left) and RLG (robust length generalization, right) across training iterations.

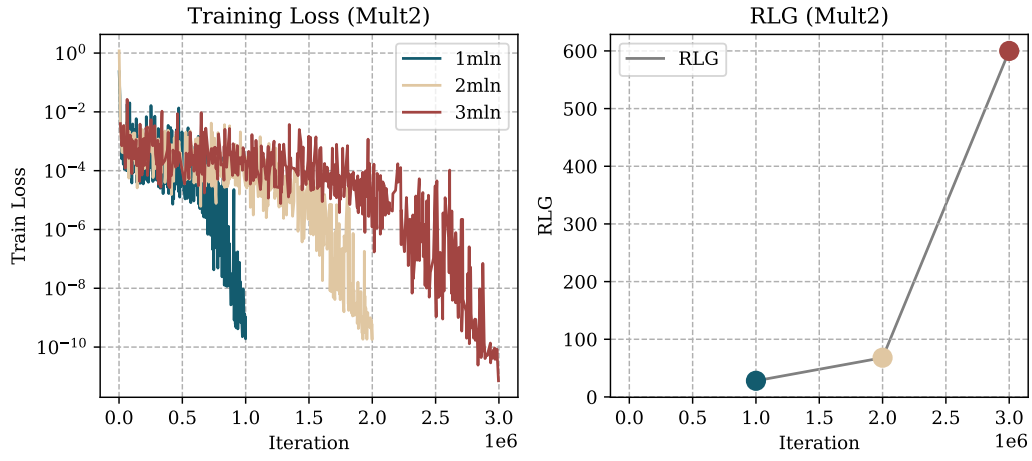


Figure 4: **Binary Multiplication:** Training loss (left) and RLG (robust length generalization, right) across training iterations.

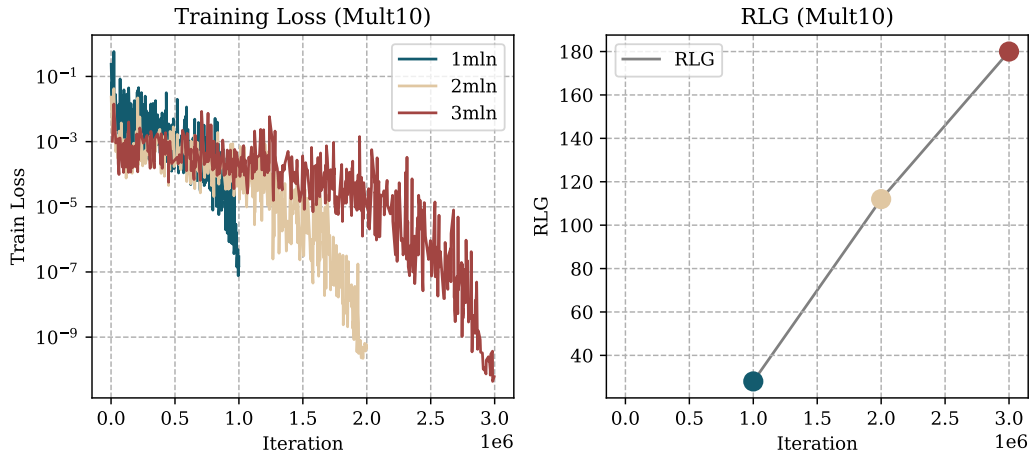


Figure 5: **Decimal Multiplication:** Training loss (left) and RLG (robust length generalization, right) across training iterations.